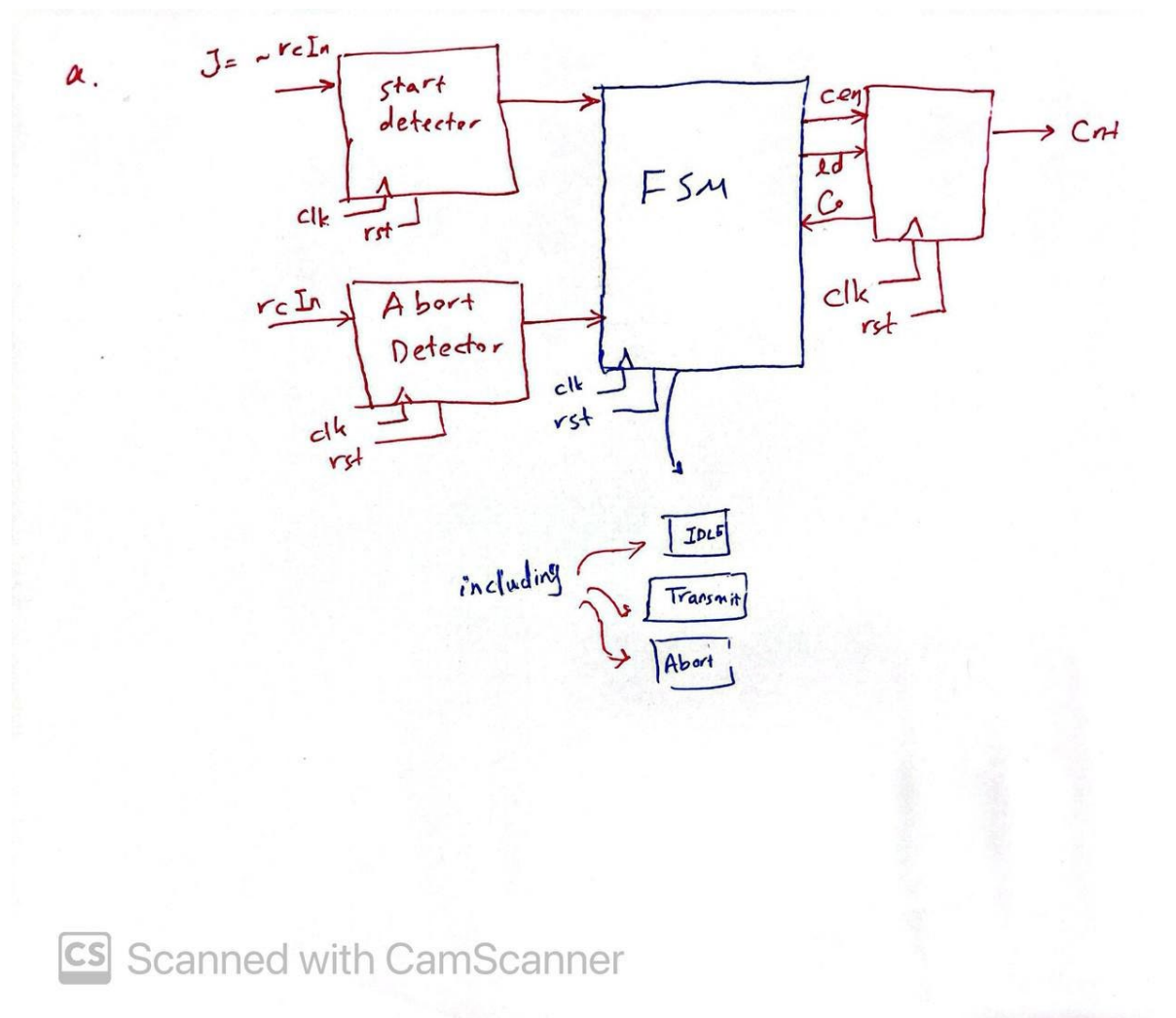
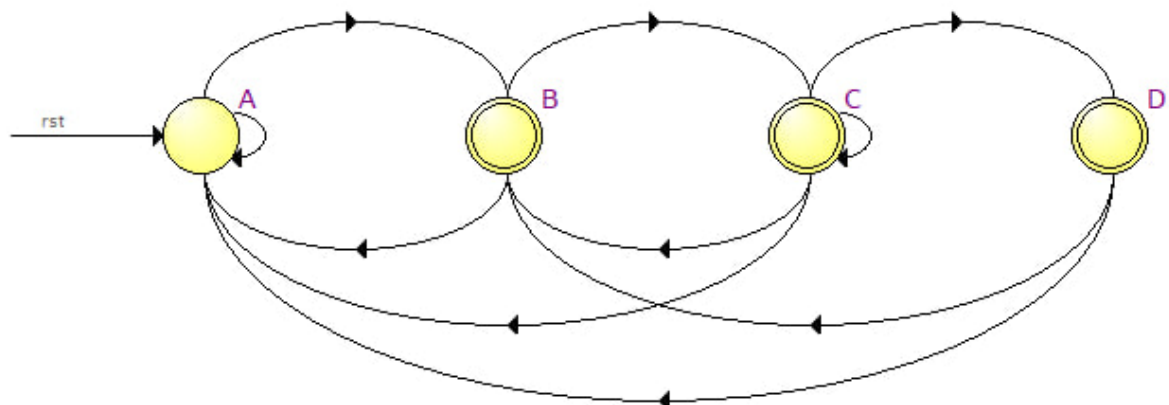


a.



b. using the state machine diagram of Quartus



	Source State	Destination State	Condition
1	A	A	(rcIn)
2	A	B	(!rcIn)
3	B	C	(rcIn)
4	B	A	(!rcIn)
5	C	D	(!cnt[0]).(cnt[1]).(cnt[2]).(!rcIn)
6	C	C	(!cnt[0]).(!cnt[1]).(rcIn) + (!cnt[0]).(cnt[1]).(!cnt[2]).(rcIn) + (cnt[0]).(rcIn)
7	C	A	(rcIn).(!cnt[0]).(cnt[1]).(cnt[2])
8	C	B	(!rcIn).(!cnt[0]).(!cnt[1]) + (!rcIn).(!cnt[0]).(cnt[1]).(!cnt[2]) + (!rcIn). (cnt[0])
9	D	A	(rcIn)
10	D	B	(!rcIn)

c.

We use the Start Detector from CA4 to detect the start sequence. For this purpose, we provide the inverted rcIn signal as input. When the start sequence is detected, the detector generates a pulse signal, which we refer to as the start_pulse. It's important to note that the CA4 detector includes a 3-bit counter as part of its internal logic.

(in part a is shown and in the paper for the presentation day.)

d.

For the Abort sequence, we also used the CA4 module. It takes the inverted rcIn signal as input, and when the Abort sequence is detected, it generates a pulse signal called abort_pulse.

2nd part:

Verilog of transmission core:

```

module transmitter_core(
    input clk,
    input rst,
    input rcIn,
    input start_transmit,
    input abort_detected,
    output logic txOut,
    output logic txValid,
    output logic txAbort
);

    parameter IDLE = 2'b00,
               DETECTED = 2'b01,
               TRANSMIT = 2'b10;

    reg [1:0] ps, ns;
    reg [6:0] tx_cnt;

```

```

always @(posedge clk or posedge rst) begin
    if (rst)
        ps <= IDLE;
    else
        ps <= ns;
end

```

```

always @(ps, start_transmit, tx_cnt, abort_detected, rcIn) begin
    ns = ps;
    txOut = 1;
    txValid = 0;
    txAbort = 0;

    case (ps)
        IDLE: begin
            if (start_transmit)
                ns = DETECTED;
            end
        DETECTED: begin
            txOut = rcIn;
            txValid = 1;
            ns = TRANSMIT;
            end
        TRANSMIT: begin
            txOut = rcIn;
            txValid = 1;
            txAbort = abort_detected;
            if (abort_detected || tx_cnt == 7'd96)
                ns = IDLE;
            end
    endcase
end

```

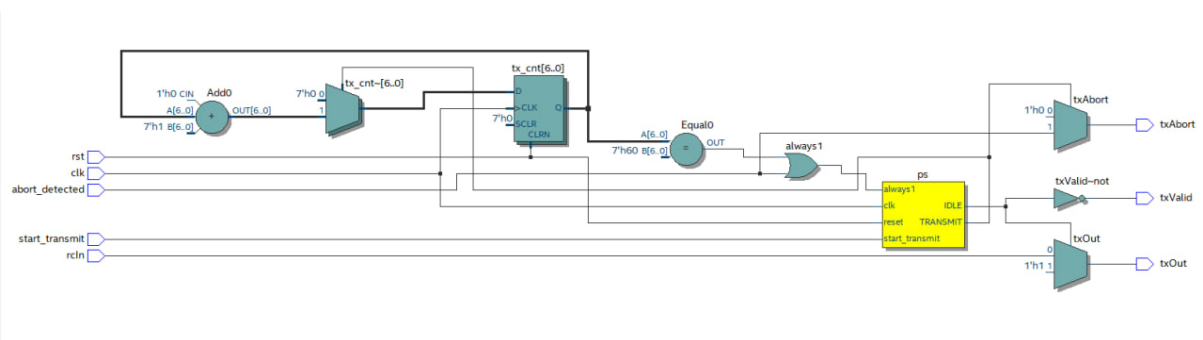
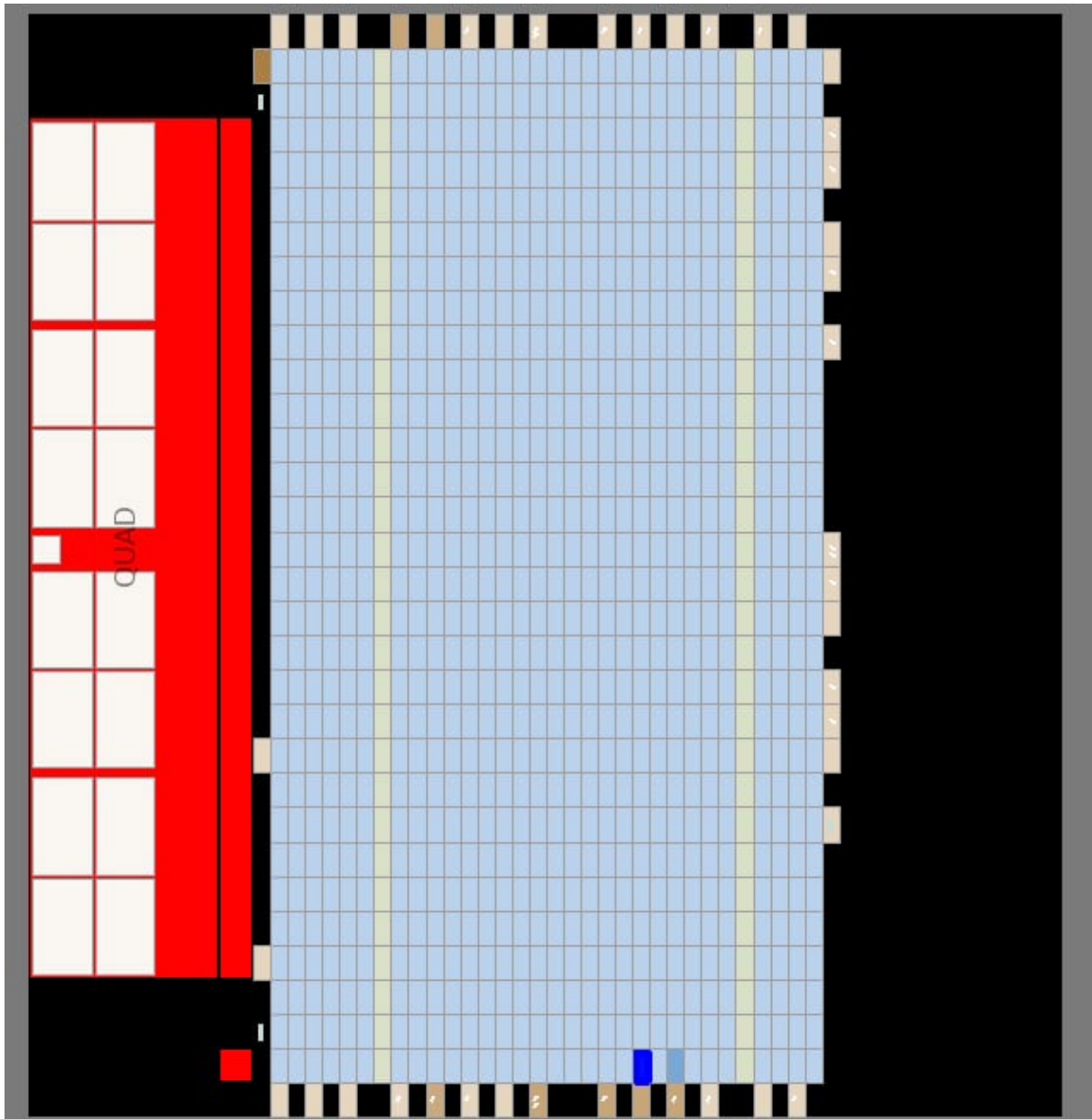
```

always @(posedge clk or posedge rst) begin
    if (rst)
        tx_cnt <= 0;
    else if (ps == TRANSMIT)
        tx_cnt <= tx_cnt + 1;
    else
        tx_cnt <= 0;
end

```

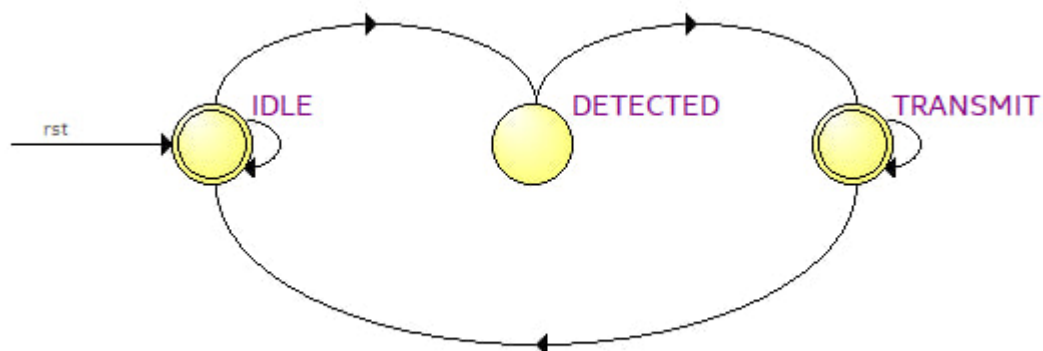
endmodule

and in Quartus we have:



State Table			
	Source State	Destination State	Condition
1	DETECTED	TRANSMIT	
2	IDLE	DETECTED	(start_transmit)
3	IDLE	IDLE	(!start_transmit)
4	TRANSMIT	TRANSMIT	(!always1)
5	TRANSMIT	IDLE	(always1)

Transitions
Encoding



3rd part

Verilog code for the ca 5 (without using other modules by using their codes into the program for using it into the Quartus)

```

`timescale 1ns/1ns
module ca5_without_m1 (
    input clk,
    input rst,
    input rcIn,
    output logic txOut,
    output logic txValid,
    output logic txAbort
);

    parameter [1:0] A = 2'b00, B = 2'b01, C = 2'b10, D = 2'b11;
    logic [1:0] ps_seq, ns_seq;
    logic [2:0] cnt;
    logic cen, ld, co;
    logic seq_detected;
    wire j = ~rcIn;
  
```

```

always @(posedge clk or posedge rst) begin
    if (rst)
        ps_seq <= A;
    else
        ps_seq <= ns_seq;
end

```

```

always @(ps_seq, j, co) begin
    ld = 0;
    cen = 0;
    ns_seq = ps_seq;
    case (ps_seq)
        A: ns_seq = j ? B : A;
        B: begin
            if (~j) begin
                ns_seq = C;
                ld = 1;
            end else
                ns_seq = A;
        end
        C: begin
            if (~j) begin
                if (~co) begin
                    ns_seq = C;
                    cen = 1;
                end else
                    ns_seq = A;
            end else begin
                if (co)
                    ns_seq = D;
                else
                    ns_seq = B;
            end
        end
        D: ns_seq = j ? B : A;
        default: ns_seq = A;
    endcase
end

```

```

always @(posedge clk or posedge rst) begin
    if (rst)
        cnt <= 3'b000;
    else begin
        if (ld)
            cnt <= 3'b001;
        else if (cen)
            cnt <= cnt + 1;
        else
            cnt <= cnt;
    end
end

```

```

always @(*) begin
    co = (cnt == 3'b110) ? 1'b1 : 1'b0;
end

```

```

assign seq_detected = (ps_seq == D);

```

```

reg [7:0] shift_reg;
wire abort_detected = (shift_reg == 8'b10000001);

always @(posedge clk or posedge rst) begin
    if (rst)
        shift_reg <= 8'b0;
    else
        shift_reg <= {shift_reg[6:0], rcIn};
end

parameter IDLE = 2'b00,
           DETECTED = 2'b01,
           TRANSMIT = 2'b10;

reg [1:0] ps, ns;
reg [6:0] tx_cnt;

always @(posedge clk or posedge rst) begin
    if (rst)
        ps <= IDLE;
    else
        ps <= ns;
end

always @(ps, seq_detected, tx_cnt, abort_detected, rcIn) begin
    ns = ps;
    txOut = 1;
    txValid = 0;
    txAbort = 0;
    case (ps)
        IDLE: begin
            txOut = 1;
            if (seq_detected)
                ns = DETECTED;
        end
        DETECTED: begin
            txOut = rcIn;
            txValid = 1;
            ns = TRANSMIT;
        end
        TRANSMIT: begin
            txOut = rcIn;
            txValid = 1;
            txAbort = abort_detected;
            if (abort_detected || tx_cnt == 7'd96)
                ns = IDLE;
        end
    endcase
end

always @(posedge clk or posedge rst) begin
    if (rst)
        tx_cnt <= 0;
    else if (ps == TRANSMIT)

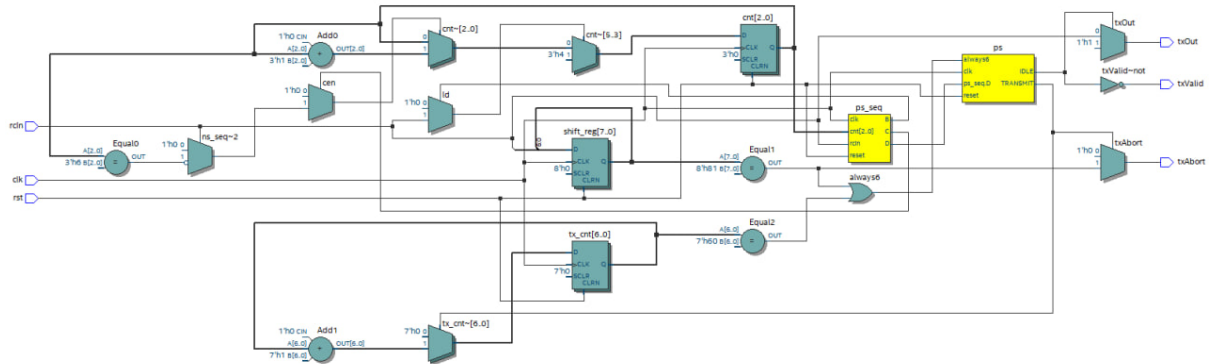
```

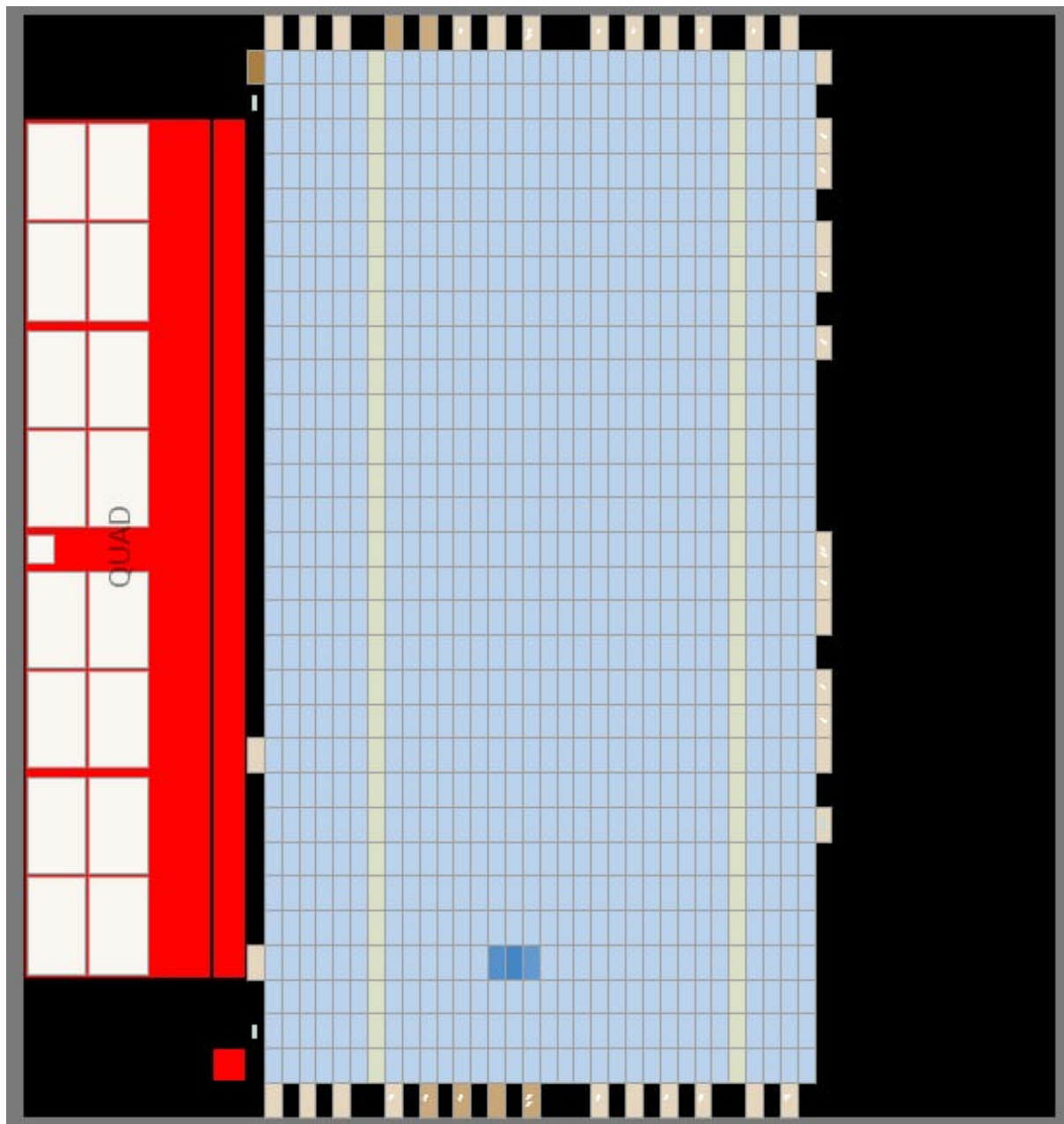
```

        tx_cnt <= tx_cnt + 1;
    else
        tx_cnt <= 0;
    end
endmodule

```

getting the post-synth ... and the test bench





The testbench and the waveform:

```
`timescale 1ns/1ns

module c5_tb;

    logic clk, rst, rcIn;
    logic txOut, txValid, txAbort;
    logic txOut0, txValid0, txAbort0;

    serial_transmitter uut (
        .clk(clk),
        .rst(rst),
        .rcIn(rcIn),
        .txOut(txOut),
        .txValid(txValid),
        .txAbort(txAbort)
    );
endmodule
```

```

);

ca5_without_m cut (
    .clk(clk),
    .rst(rst),
    .rcIn(rcIn),
    .txOut(txOut0),
    .txValid(txValid0),
    .txAbort(txAbort0)
);

always #5 clk = ~clk;

task send_bits(input [7:0] bits);
    integer i;
    begin
        for (i = 7; i >= 0; i = i - 1) begin
            rcIn = bits[i];
            #10;
        end
    end
endtask

initial begin

    clk = 0;
    rst = 1;
    rcIn = 1;
    #20;

    rst = 0;
    #20;

    $display("Sending START sequence: 01111110");
    send_bits(8'b01111110);

    #200;

    $display("Sending ABORT sequence: 10000001");
    send_bits(8'b10000001);

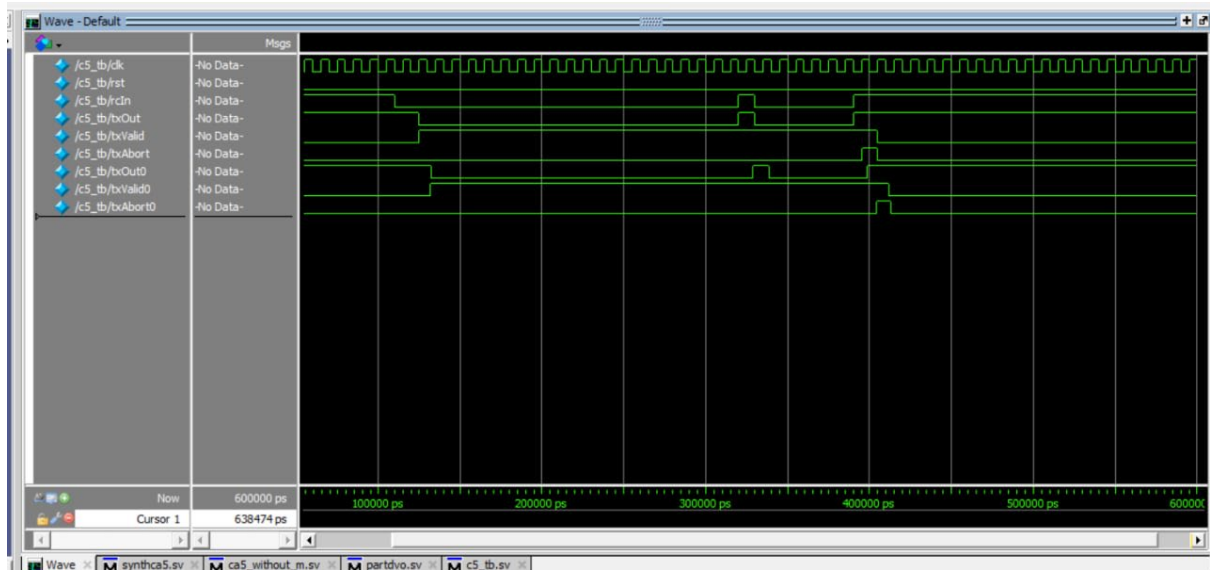
    #200;

    $display("Test complete.");
    $stop;
end

endmodule

```

and the waveform is:



This is the waveform for both pre synthesized and post synthesized version and we can see the delay in it .It works properly.